

Parcial 2

Objetivo: Demostrar el funcionamiento de una aplicación Fullstack (FastAPI + React) que integre persistencia de datos, relaciones complejas, gestión de estado de servidor y navegación.

FECHA DE ENTREGA: 25-05-26

INTEGRANTES POR GRUPO: 4

Requerimientos del Video

- **Duración máxima:** 15 minutos.
- **Formato:** Grabación de pantalla con voz en off (pueden usar herramientas como Loom, OBS o Clipchamp).
- **Contenido:** El video debe dividirse en tres secciones: Arquitectura Backend, Interfaz Frontend y Demo de Flujo Completo.
- Deberá de presentarse cada integrante.
- **El Video y los repositorios tienen que ser públicos si no serán desaprobados**

Estructura de la Presentación

1. Backend: El Corazón de la API (4-5 minutos)

- **Módulos a realizar:** Pedido, DetallePedido, EstadoPedido, FormaPago, Usuario, Rol, UsuarioRol, DireccionEntrega
- **Modelado de Datos:** Muestra tus clases de SQLAlchemy. Explica cómo implementaste las relaciones, mencionando el uso de Relationship y back_populates.
- **Seed:** Se deberá mostrar que el seed obligatorio funcione correctamente.
- **Endpoints y Lógica:** Muestra un endpoint que utilice Annotated y Query para filtros o paginación. Explica brevemente cómo manejas las excepciones (HTTPException) y los códigos de estado.
- **Persistencia:** Breve vistazo a la conexión con PostgreSQL y cómo se ven las tablas reflejadas (puedes mostrar pgAdmin o DBeaver).

Frontend: Experiencia de Usuario y Estado (4-5 minutos)

- **Módulos a realizar:**

Frontend 2 proyectos consumen un backend

Módulo Store

- **Home store**
- **Carrito** (persistencia localStorage, middleware persist)
- **Pantalla de pedidos**
- Realizar pedido (sin pasarela de pago)
- Instancia de axios
- Axios interceptor
- Rutas
- **Server State (TanStack Query):** Explica una implementación de useQuery para el listado y una useMutation para el alta o edición. Muestra dónde haces la

- **Navegación:** Muestra la configuración de react-router-dom y cómo pasas parámetros dinámicos a través de la URL (ej. el ID para ver el detalle).
- **Estructura de módulos**

Módulo Administración

- Inicio de sesión
- Token cookie onlyHttp
- Protección de rutas por autenticación y por rol
- Instancia de axios
- Axios interceptor
- **Pantalla categoría** y subcategoría del parcial 1
- **Pantalla ingrediente** del parcial 1
- **Pantalla productos** del parcial 1
- Modo Admin puede realizar acciones Modo empleado solo puede ver
- **Pantalla empleado** cajero
 - Cambia estados del pedido Aprobado, en proceso, listo, entregado
- **Estructura y Tipado:** Muestra un componente clave y su respectiva interfaz en TypeScript para las Props.
- **Server State (TanStack Query):** Explica una implementación de useQuery para el listado y una useMutation para el alta o edición. Muestra dónde haces la **invalidación de la caché** (invalidateQueries).
- **Navegación:** Muestra la configuración de react-router-dom y cómo pasas parámetros dinámicos a través de la URL (ej. el ID para ver el detalle).
- **Estructura de módulos**

Demo en Vivo: El Flujo Integrador (5 minutos)

- **CRUD Completo:** Crea un nuevo registro, editalo y elimínalo mientras muestras la consola del navegador o la terminal del backend para validar las peticiones.
- **Validaciones:** Intenta cargar datos inválidos para demostrar que las validaciones de Pydantic y los mensajes de error en el Frontend funcionan correctamente.
- **Relaciones en la UI:** Muestra cómo se visualizan los datos relacionados (ej. "Este producto pertenece a la categoría X y tiene las etiquetas A y B").

Criterios de Evaluación

1. **Claridad Técnica:** Uso correcto de la terminología vista en clase.
2. **Integración:** El frontend debe consumir datos reales del backend (no *mock data*).
3. **Diseño:** Aplicación coherente de Tailwind CSS para una interfaz limpia y responsive.
4. **Resolución de Problemas:** Explicación de algún desafío técnico que encontraron y cómo lo resolvieron.

Back-End

1. Autenticación y Autorización (/api/v1/auth/)

- Registro de usuarios con asignación automática del rol CLIENT
- Login con email/password → genera cookie “access token” (JWT, 30 min)
- Endpoint GET /me para obtener datos del usuario autenticado

2. Sistema de Roles (RBAC)

Cuatro roles con permisos diferenciados:

Rol	Código	Capacidades
Administrador	ADMIN	CRUD completo de todo el sistema
Gestor de Stock	STOCK	Leer productos, actualizar stock y disponibilidad
Gestor de Pedidos	PEDIDOS	Ver y avanzar estados de pedidos
Cliente	CLIENT	Catálogo, carrito, pedidos propios

3. Catálogo — Categorías (/api/v1/categorias/)

- CRUD completo (solo ADMIN)
- Categorías jerárquicas con autorreferencia (parent_id) y consulta recursiva
- Soft delete convalidación: no se puede eliminar si tiene productos activos (HTTP 409)
- Listado público con filtro por parent_id y paginación

4. Catálogo — Productos (/api/v1/productos/)

- Listado público con filtros: categoría, disponibilidad, búsqueda por texto, paginación
- CRUD completo (solo ADMIN)
- Gestión de ingredientes asociados al producto, incluyendo el campo es_alergeno
- PATCH /disponibilidad para activar/desactivar un producto (ADMIN y STOCK)
- Soft delete
- Campos de stock directamente en el modelo de Producto: stock_cantidad y disponible

5. Gestión de Pedidos (/api/v1/pedidos/)

- Creación de pedido desde el carrito con transacción atómica (Unit of Work)
- Máquina de estados de 6 estados: PENDIENTE → CONFIRMADO → EN_PREP → EN_CAMINO → ENTREGADO / CANCELADO
- Avance de estado validado en la capa de servicio (nunca en el router)
- Audit Trail append-only: tabla HistorialEstadoPedido con solo INSERTs, jamás UPDATE/DELETE
- Snapshot Pattern: precio y nombre del producto se guardan inmutables al crear el pedido
- Historial completo de transiciones ordenado por fecha
- Cancelación por el propio cliente (solo desde PENDIENTE o CONFIRMADO)
- Listado: el CLIENT ve solo sus pedidos; ADMIN/PEDIDOS ven todos

7. Direcciones de Entrega (/api/v1/direcciones/)

- CRUD completo para el usuario autenticado
- PATCH /principal para marcar una dirección como principal (solo una por usuario)
- Soft delete
- Campo alias (ej: "Casa", "Trabajo")

8. Panel de Administración (/api/v1/admin/)

- Gestión de usuarios: listado paginado con filtro por rol, actualización, soft delete y asignación de roles

9. Arquitectura y patrones del backend

Patrón	Descripción
Unit of Work	Gestión transaccional atómica: commit/rollback automático. El Service nunca hace session.commit() directamente
Repository Pattern	BaseRepository[T] genérico con CRUD base; cada módulo extiende con sus queries propias
Service Layer	Lógica de negocio stateless. Nunca en el router
Soft Delete	deleted_at TIMESTAMPTZ en todas las entidades de negocio

Snapshot Pattern	Precio y nombre inmutables en DetallePedido
Audit Trail Append-Only	HistorialEstadoPedido solo permite INSERTs

10. Configuración y seguridad

- CORS con CORSMiddleware
- bcrypt para hash de contraseñas (cost factor ≥ 12)
- JWT con PyJWT: access token (30 min)
- Seed data obligatorio: roles, estados de pedido, formas de pago y usuario admin por defecto
- API REST documentada automáticamente en /docs (Swagger UI) y /redoc